

Setting Up a Multi-Node Kubernetes Cluster in OrbStack

<https://gemini.google.com/app/12610624a341aafb>

User prompt: *i want some hand-On experience on Kubernetes, so i planned why not setup my own k8s cluster with 1 control plane node and 2 worker node inside orbstack in my macbook. give me an overall plan of how to proceed with this plan inside orbstack.*

Response: OrbStack provides a built-in single-node Kubernetes cluster. To run a **3-node cluster (1 control plane + 2 workers)**, you have two primary approaches depending on your learning objective:

- **Path A: Kind (Kubernetes in Docker) on OrbStack** — Nodes run as Docker containers managed by OrbStack's Docker engine. Spin-up takes under a minute, uses minimal RAM, and is ideal for learning workloads, scheduling, DaemonSets, and Helm.
- **Path B: OrbStack Linux Machines + kubeadm (or k3s)** — Spin up 3 distinct lightweight Linux machines (k8s-cp, k8s-node1, k8s-node2). You configure containerd, networking, CNI, and join tokens manually. This mirrors real-world production or CKA exam environments.

Path A: Kind Multi-Node (Fastest & Lightest)

1. Prerequisites

- Ensure OrbStack is running with Docker enabled.
- Install kind and kubectl on your Mac:

```
brew install kind kubectl
```

2. Define the 3-Node Cluster Topology

Create a config file named `cluster-config.yaml`:

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
  - role: worker
```

3. Spin Up the Cluster

```
kind create cluster --name dev-cluster --config cluster-config.yaml
```

4. Verification Run:

```
kubectl get nodes -o wide
```

You will see `dev-cluster-control-plane`, `dev-cluster-worker`, and `dev-cluster-worker2` in a Ready state.

Path B: OrbStack Linux Machines + Kubeadm (CKA / Infrastructure Route)

Phase 1: Spin Up the 3 Linux Machines OrbStack creates lightweight Linux environments sharing macOS's virtualization layer in seconds without pre-allocating gigabytes of RAM:

```
orb create ubuntu:noble k8s-cp
orb create ubuntu:noble k8s-node1
orb create ubuntu:noble k8s-node2
```

Phase 2: Base System & Runtime Setup (Run on all 3 nodes) SSH into each machine (`orb -m k8s-cp`, etc.) to prepare the node:

1. **Network forwarding & modules:** Enable overlay and br_netfilter kernel modules, and set `net.ipv4.ip_forward = 1` in `sysctl`.
2. **Container Runtime:** Install containerd and configure the systemd cgroup driver (`SystemdCgroup = true` in `/etc/containerd/config.toml`).
3. **K8s Packages:** Install kubelet, kubeadm, and kubectl using the official Kubernetes APT repository.

Phase 3: Control Plane Initialization On `k8s-cp`:

1. Find its internal IP (`ip a` or via OrbStack GUI/CLI).
2. Initialize:

```
sudo kubeadm init --pod-network-cidr=10.244.0.0/16 --apiserver-advertise-address=<CP_NODE_IP>
```

3. Set up the local kubeconfig for the control plane user:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

4. Deploy a Container Network Interface (CNI), such as Flannel or Calico:

```
kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
```

Phase 4: Join Worker Nodes Copy the `kubeadm join ... --token ... --discovery-token-ca-cert-hash ...` string generated during Phase 3 and execute it with `sudo` inside `k8s-node1` and `k8s-node2`.

Phase 5: Connect from your Mac Copy `/etc/kubernetes/admin.conf` from `k8s-cp` to your Mac's `~/kube/config` to manage the entire cluster directly using your Mac's native terminal and `kubectl`.

Sources

1. [Kubernetes - OrbStack Docs](#)
2. [OrbStack for Kubernetes: Maximum Learning, Minimum RAM](#)

User prompt: i think i will learn more things about setting up a k8s cluster from scratch in path B. give me a overall plan of how to proceed with that. dont give me exact command to type, i'll ask you if i need the exact command, for now just give me detailed plan of how i can proceed and what are the prerequisites. i want to setup every component in the k8s cluster(kube proxy, kube scheduler, etcd, csi, cni everything, basically a full fledged working k8s cluster).

Response: Building a multi-node cluster from the OS layer up across separate Linux virtual machines will give you a concrete mental model of node topology, networking, systemd daemon lifecycles, and cluster storage.

Here is the architectural plan and operational sequence to get every component running from scratch.

Phase 0: Architecture & Prerequisites

Prerequisites on your Mac:

- **OrbStack:** Installed and healthy.
- **Mac Host Tools:** `kubectl` installed locally (via Homebrew) so you can administer the cluster directly from macOS without staying SSH'd into the VMs.
- **Host Resources:** Allocate at least 4 CPU cores and 6–8 GB of dynamic memory across your VMs in OrbStack preferences.

Cluster Node Architecture:

Node Name	Role	Core Services / Responsibilities
<code>k8s-cp</code>	Control Plane	<code>etcd</code> , <code>kube-apiserver</code> , <code>kube-controller-manager</code> , <code>kube-scheduler</code> , CNI agent, <code>kube-proxy</code> , <code>kubelet</code>
<code>k8s-worker-1</code>	Worker Node	<code>kubelet</code> , <code>kube-proxy</code> , CNI agent, CSI node plugin, user Pod workloads
<code>k8s-worker-2</code>	Worker Node	<code>kubelet</code> , <code>kube-proxy</code> , CNI agent, CSI node plugin, user Pod workloads

Phase 1: Virtual Machine Provisioning & Host Preparation

1. Create 3 Linux Machines:

- Spin up three clean, identical Ubuntu instances (`k8s-cp`, `k8s-worker-1`, `k8s-worker-2`).

2. Network Topology & Discovery:

- Note the static internal IP addresses assigned to each machine by OrbStack.
- Update `/etc/hosts` on all three machines so they can reliably resolve each other by hostname without depending on external DNS.

3. OS-Level Kernel Configuration (All 3 nodes):

- **Swap:** Ensure swap is completely disabled (Kubernetes memory management requires swap to be off).
- **Kernel Modules:** Load `overlay` (for container filesystem layering) and `br_netfilter` (so bridged packets traverse iptables rules).
- **Sysctl Network Knobs:** Enable `net.bridge.bridge-nf-call-iptables`, `net.bridge.bridge-nf-call-ip6tables`, and `net.ipv4.ip_forward`.

Phase 2: Container Runtime Installation (All 3 nodes)

Every node needs a CRI-compliant container runtime before Kubernetes binaries can run.

1. Install Containerd:

- Install the standard `containerd` package from official repositories.

2. Configure Cgroup Driver:

- Generate the default `containerd` configuration.
- Set the cgroup driver to `systemd` (critical for matching `kubelet`'s cgroup driver to avoid node instability).

3. Start the Service:

- Enable and start `containerd` via `systemctl`.

Phase 3: Core Kubernetes Binaries (All 3 nodes)**1. Add Official Kubernetes Repositories:**

- Install package dependencies (`apt-transport-https`, `ca-certificates`, `curl`, `gpg`).
- Add the official community package repository key and repository list.

2. Install Binaries:

- Install matching versions of `kubelet`, `kubeadm`, and `kubectl`.
- Mark/hold the packages to prevent accidental updates from package managers while learning.

3. Enable Kubelet:

- Enable `kubelet` service (it will continuously restart/wait in the background until `kubeadm` configures its manifests and certificates).

Phase 4: Control Plane Initialization (k8s-cp)

This step brings up the core brains of the cluster.

1. Define Pod Network CIDR & API Endpoint:

- Select a dedicated, non-overlapping subnet for Pod IPs (e.g., `10.244.0.0/16` for Flannel, or `192.168.0.0/16` for Calico).

2. Run Initialization:

- Execute the initialization specifying the node's advertise IP and your chosen Pod CIDR.
- `kubeadm` automatically:
 - Generates CA authorities, client/server TLS certificates, and service account keypairs.
 - Generates static pod manifests under `/etc/kubernetes/manifests/` for:
 - `etcd`: The distributed key-value store.
 - `kube-apiserver`: The central REST API gateway.
 - `kube-controller-manager`: Reconciles desired vs. actual states.
 - `kube-scheduler`: Assigns unscheduled pods to nodes.
 - Deploys the cluster-wide `kube-proxy` DaemonSet and CoreDNS ConfigMap/Deployment.

3. Configure Access (kubeconfig):

- Copy `admin.conf` to your local non-root user directory on `k8s-cp`.
- Copy this file to your macOS host (`~/ .kube/config`) so your local terminal has administrative cluster access.

Phase 5: Cluster Networking (CNI Implementation)

Until a Container Network Interface is deployed, CoreDNS pods remain in Pending and all nodes report NotReady.

1. Select a CNI Plugin:

- **Flannel**: Simple, lightweight overlay network (VXLAN). Great for straightforward setups.
- **Calico**: Production-grade, supports NetworkPolicies, BGP, and advanced routing.
- **Cilium**: eBPF-based, high performance, can optionally replace `kube-proxy`.

2. Deploy the CNI:

- Apply the CNI manifest to the cluster.
- Verify that CNI DaemonSet pods launch on `k8s-cp`, CoreDNS transitions to Running, and the control plane node shifts to Ready.

Phase 6: Joining Worker Nodes & Verifying kube-proxy**1. Execute Join Command:**

- Run the secure bootstrap join command (with the generated token and CA hash) on `k8s-worker-1` and `k8s-worker-2`.

2. Validate Node Registration:

- From your Mac host, check that all three nodes transition from NotReady to Ready.

3. Verify Node-Level Networking:

- Verify that each worker node runs:
 - An active instance of `kubelet` managing local container runtimes.
 - A `kube-proxy` Pod (managing iptables/IPVS rules for cluster services).
 - A CNI Pod agent providing cross-node pod-to-pod routing.

Phase 7: Dynamic Storage (CSI Implementation)

A production-ready cluster requires a Container Storage Interface to dynamically provision persistent storage for stateful workloads.

1. Select a Local CSI Provider:

- **Rancher Local Path Provisioner:** Extremely lightweight, dynamically provisions `hostPath` volumes on the underlying VM disk. Ideal for homelabs and local multi-node setups.
- **Longhorn / Rook-Ceph:** Distributed block storage across workers (heavier, requires dedicated raw disks/partitions).

2. Deploy & Configure:

- Deploy the CSI controller and node driver DaemonSets.
- Set up a default `StorageClass` with dynamic provisioning enabled.

3. Verification:

- Create a test `PersistentVolumeClaim` (PVC) and ensure it binds automatically when mounted by a stateful test Pod.

Phase 8: Cluster Health & End-to-End Validation Checklist

- [] **Node Status:** All 3 nodes report `STATUS: Ready`.
- [] **System Pods:** `kube-system` pods (`etcd`, `apiserver`, `controller-manager`, `scheduler`, `coredns`, `kube-proxy`, `CNI daemonset`) are all healthy and restart-free.
- [] **Pod-to-Pod Communication:** A Pod on `k8s-worker-1` can ping/curl a Pod on `k8s-worker-2` across the overlay network.
- [] **Cluster DNS:** Pods can resolve internal services via CoreDNS (e.g., `kubernetes.default.svc.cluster.local`).
- [] **Service Routing:** A ClusterIP and NodePort service route traffic evenly to backend Pods on worker nodes.
- [] **Persistence:** A test deployment with a PVC writes data to a volume, survives a Pod deletion, and re-attaches to the new replica intact.

Whenever you are ready to tackle Phase 1 and 2, let me know and we can dive into the exact preparations.

User prompt: *what to select here*

Response: Select **Linux**.

Here is why:

- **Linux (Recommended):** Gives you full access to lightweight virtual machines so you can spin up the three separate instances (`k8s-cp`, `k8s-worker-1`, `k8s-worker-2`) and manually install `containerd`, `kubeadm`, and every cluster component from scratch.
- **Kubernetes (Do NOT select for this):** OrbStack's built-in Kubernetes is a pre-packaged, single-node cluster that manages everything automatically under the hood, bypassing the scratch setup experience.
- **Docker:** Only provisions OrbStack's Docker engine container runtime.

Select **Linux** (or choose both **Docker** and **Linux** if the installer allows multi-selection) to proceed to the machine setup.

User prompt: *here?*

Response: Fill in these values for your first machine (the control plane):

- **Name:** Change `ubuntu` to `k8s-cp` (this makes it distinct from your worker nodes).
- **Distribution:** Keep as **Ubuntu**.
- **Version:** Select **24.04 LTS (Noble Numbat)** or **22.04 LTS (Jammy Jellyfish)** if available in the dropdown. (The default shown, `26.04`, is a development release; standard LTS releases like `24.04` or `22.04` have guaranteed, stable package repositories for `kubelet` and `containerd`).
- **Architecture:** Keep as **arm64** (native Apple Silicon performance).
- **Username:** Keep default or your preferred local user (e.g., `macbee`).

Click **Create**.

Once created, you will repeat this exact process two more times for the worker nodes, setting their names to `k8s-worker-1` and `k8s-worker-2`.

If you'd like a quick visual overview of setting up and exploring Ubuntu 24.04 environments, check out [Installing Ubuntu 24.04 "Noble Numbat" and Its Features](http://googleusercontent.com/youtube_content/1). This walkthrough demonstrates the base setup and characteristics of the Noble Numbat release.

http://googleusercontent.com/youtube_content/1

User prompt: how to do phase 1.3.1 task? (Swap: Ensure swap is completely disabled (Kubernetes memory management requires swap to be off)).

Response: To handle swap for Kubernetes, you need to check its active state and make sure it does not persist across reboots.

In OrbStack virtual machines, swap is typically disabled by default, but you should still verify and enforce it on all three nodes.

Step 1: Check Active Swap

Run the following command in your terminal inside the machine:

```
free -m
```

Look at the **Swap** row:

- If total, used, and free all show 0, swap is already disabled.
- If you see numbers greater than 0, proceed to turn it off.

Alternatively, run:

```
swapon --show
```

If it returns completely blank output, no swap device or swap file is active.

Step 2: Disable Swap Immediately (Runtime)

If swap was active, disable it instantly without rebooting:

```
sudo swapoff -a
```

To verify: run `free -m` again. The Swap: line should now read 0 0 0.

Step 3: Prevent Swap from Re-enabling on Boot

Linux systems mount swap devices defined in the filesystem table (`/etc/fstab`).

1. Open `/etc/fstab` with a text editor:

```
sudo nano /etc/fstab
```

2. Look for any line that contains the word `swap` (e.g., `/swap.img none swap sw 0 0` or a swap partition UUID).

3. Add a `#` at the very beginning of that line to comment it out:

```
# /swapfile none swap sw 0 0
```

4. Save and exit (in nano, press `Ctrl + O`, Enter, then `Ctrl + X`).

(Note: OrbStack's lightweight Ubuntu images usually have a near-empty `/etc/fstab` with no swap configured at all. If there is no line referencing swap, no edits are necessary.)

User prompt: what is 1.3.2, why it is used and how to do that?

Response: **Task 1.3.2** is enabling two essential Linux kernel modules on every node: `overlay` and `br_netfilter`.

What Are They and Why Are They Needed?

- **overlay (OverlayFS):**
 - **What it is:** A union filesystem module built into the Linux kernel that combines multiple directory trees into a single merged view.
 - **Why it is needed:** Container runtimes like `containerd` use `overlayfs` as the default storage driver. It allows images to share read-only base layers while giving each running container a thin, writable top layer. Without this module, `containerd` cannot manage container images or spin up containers efficiently.
- **br_netfilter (Bridge Netfilter):**
 - **What it is:** A kernel module that allows network traffic moving across virtual ethernet bridges (Layer 2) to be inspected and filtered by `iptables` and `netfilter` rules (Layer 3/4).

- **Why it is needed:** Kubernetes networking models rely on virtual bridges and interfaces to connect pods. For cluster networking, network policies, and kube-proxy service routing to work properly, bridge traffic must pass through the host's firewall rules. Without `br_netfilter`, pod-to-pod and service traffic will either drop silently or fail to be routed.

How to Do It

You will configure these modules so they load automatically at system boot, and load them into the current running kernel session immediately.

1. Configure modules to load on boot

Create a configuration file in `/etc/modules-load.d/`:

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
```

2. Load modules into the current session

Run `modprobe` to load them immediately without needing to restart the machine:

```
sudo modprobe overlay
sudo modprobe br_netfilter
```

3. Verification

Verify that both modules are actively loaded into the running kernel:

```
lsmod | grep -E "overlay|br_netfilter"
```

Verification check: If the step succeeded, the output will list both `overlay` and `br_netfilter` along with memory/usage details. If the output is empty, the modules failed to load.

(Note: Repeat this step on all 3 nodes: `k8s-cp`, `k8s-worker-1`, and `k8s-worker-2`.)

User prompt: `macbeee@k8s-cp:/etc/modules-load.d$ lsmod | grep -E "overlay|br_netfilter"` macbeee@k8s-cp:/etc/modules-load.d\$

Response: Inside OrbStack machines, Linux runs on an optimized custom kernel where features are frequently built directly into the kernel (builtin) rather than compiled as loadable dynamic modules (`.ko`). When a feature is compiled directly into the kernel, `lsmod` (which only reports dynamically loaded modules) outputs nothing.

Check if they are already built into your running kernel:

```
sudo modprobe overlay
sudo modprobe br_netfilter
```

If `modprobe` exits silently with return code 0 (no errors), check the kernel config or `sysfs` entries directly:

```
# Check if bridge netfilter sysctl entries exist
ls /proc/sys/net/bridge/
```

Or check the active kernel config:

```
grep -E "CONFIG_OVERLAY_FS|CONFIG_BRIDGE_NETFILTER" /boot/config-$(uname -r) 2>/dev/null || zgrep -E "CONF
```

- **If you see `=y`:** The feature is built directly into the kernel image. It is permanently active and ready.
- **If you see `=m`:** It is a loadable module and `sudo modprobe <name>` will load it.
- **If `/proc/sys/net/bridge` exists:** `br_netfilter` capabilities are operational.

If `sudo modprobe overlay` and `sudo modprobe br_netfilter` ran with zero errors, keep `/etc/modules-load.d/k8s.conf` in place and move directly to **Phase 1.3.3 (sysctl configuration)**.

User prompt: what is 1.3.3, why it is used and how to do that?

Response: **Task 1.3.3** is configuring `sysctl` kernel networking parameters to allow packet routing and firewall filtering for containers.

What Are These Parameters and Why Are They Needed?

You configure three main parameters:

- **`net.ipv4.ip_forward = 1`**
 - **What it does:** Allows the Linux kernel to act as a router and forward network packets between different network interfaces.
 - **Why it is needed:** Without this, your node can only receive packets destined for its own IP address. Containers live in separate virtual network namespaces with their own IP ranges (Pod CIDR). When a container sends a packet to the outside world or to another node, the host must forward that traffic. If set to 0, all cross-namespace and outbound container traffic drops.

- **net.bridge.bridge-nf-call-iptables = 1**
 - **What it does:** Forces network traffic crossing a Linux virtual ethernet bridge (Layer 2) to be passed to iptables (Layer 3/4) for filtering and rule processing.
 - **Why it is needed:** Kubernetes CNI plugins and kube-proxy rely on iptables chains to implement cluster-internal routing, NetworkPolicies, and Service port translations. Without this enabled, bridge traffic bypasses iptables entirely, breaking service discovery and security policies.
- **net.bridge.bridge-nf-call-ip6tables = 1**
 - **What it does:** The IPv6 equivalent of the bridge iptables call.
 - **Why it is needed:** Ensures IPv6 bridged traffic is passed to ip6tables to prevent network rule bypasses.

How to Do It

1. Create a persistent sysctl configuration file

Place the parameters in /etc/sysctl.d/ so they persist across machine restarts:

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF
```

2. Apply the parameters immediately

Load the settings from your new configuration file into the active running kernel without rebooting:

```
sudo sysctl --system
```

(You will see output showing various .conf files loading, concluding with /etc/sysctl.d/k8s.conf.)

3. Verification

Verify each value directly from the running kernel:

```
sysctl net.ipv4.ip_forward net.bridge.bridge-nf-call-iptables net.bridge.bridge-nf-call-ip6tables
```

Verification check: The output must match:

```
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
```

(Note: Apply this configuration on all 3 nodes: k8s-cp, k8s-worker-1, and k8s-worker-2.)

User prompt: my phase 1 is completed. i want to install in such a way that after doing the complete in-house setup also want to upgrade the whole cluster(1 cp and 2 wp) without downtime, which will help me to learn one more skill of upgrading the k8s cluster without downtime. so tell me right now which version of k8s to setup so that i can upgrade it as well

Response: To learn the zero-downtime rolling upgrade workflow, install **Kubernetes v1.34** now, with the goal of upgrading to **v1.35**.

Why This Pair?

- **Strict Version Skew Policy:** Kubernetes only supports minor upgrades one version at a time ($1.N \rightarrow 1.N + 1$). You cannot jump directly from 1.33 $\rightarrow$ 1.35.
- **Ecosystem Compatibility:** Both v1.34 and v1.35 are fully supported and stable across major CNIs (Flannel, Calico) and containerd.
- **Realistic Upgrade Path:** Installing v1.34 gives you an active, modern cluster immediately, while keeping you exactly one step away from practicing the standard production upgrade sequence.

Zero-Downtime Architecture with 1 CP and 2 Workers

To achieve zero workload downtime during the upgrade:

1. **Multi-Replica Workloads:** Run workloads as a Deployment with at least 2 replicas and a PodDisruptionBudget (minAvailable: 1).
2. **Pod Anti-Affinity:** Ensure replicas are spread across both worker nodes (k8s-worker-1 and k8s-worker-2) so they never land on the same node simultaneously.
3. **Rolling Node Drains:**
 - Upgrade the control plane (k8s-cp) first. The control plane API briefly restarts, but running worker pods continue serving traffic uninterrupted.
 - Cordon and drain k8s-worker-1. Workloads gracefully evict and reschedule onto k8s-worker-2.

- Upgrade binaries on `k8s-worker-1`, uncordon it.
- Repeat the drain, upgrade, and uncordon sequence on `k8s-worker-2`.

Version Pinning Strategy for Phase 2 & 3

When you add the Kubernetes package repository in the next phase, you will pin the APT package versions to:

- **Target Initial Version:** `1.34.x` (e.g., `1.34.0` or latest `1.34` patch)
- **Target Upgrade Version (Later):** `1.35.x`

In the package manager, you will lock these with `apt-mark hold kubelet kubeadm kubect1` so an accidental `apt upgrade` doesn't bump them ahead of time.

User prompt: what is 2.1.1, why it is used and how to do it?

Response: **Phase 2.1.1 is installing the containerd runtime package** on your Linux machines from the official repository.

What Is It and Why Is It Used?

- **What it is:** containerd is an open-source, industry-standard container runtime (graduated CNCF project). It manages the entire container lifecycle on the host: pulling images from registries, unpacking them, executing containers via `runc`, managing storage, and handling low-level process attachment.
- **Why it is used:** Kubernetes does not directly run containers itself. The Kubernetes node agent (`kubelet`) talks to a runtime using the **Container Runtime Interface (CRI)** API via a Unix domain socket. In modern Kubernetes, containerd is the most widely adopted CRI runtime because it is lightweight, decoupled from the full Docker daemon, and purpose-built for orchestration environments.

How to Do It

Perform these steps on all 3 nodes (`k8s-cp`, `k8s-worker-1`, `k8s-worker-2`).

1. Install prerequisite utilities

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg
```

2. Add the official repository GPG key

The `containerd.io` package is officially maintained and packaged through Docker's repository:

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

3. Add the APT repository

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.dock
```

4. Install containerd.io

Update your package index and install only `containerd.io` (do not install `docker-ce`):

```
sudo apt-get update
sudo apt-get install -y containerd.io
```

5. Verification

Verify that the binary exists and prints its version:

```
containerd --version
```

Verification check: The command should return the installed containerd version string without errors.

User prompt: what is 2.2, why it is used and how to do that?

Response: **Phase 2.2 is generating the containerd configuration file and setting its cgroup driver to systemd.**

What Is It and Why Is It Used?

- **What are cgroups (Control Groups)?** Linux cgroups allocate and throttle hardware resources (CPU, memory, I/O) among running processes and containers.
- **Why systemd as the cgroup driver?** On systemd-based Linux distributions (like Ubuntu), systemd acts as the root init process and manages its own unified cgroup tree. By default, containerd's low-level container runtime (`runc`) defaults to the `cgroupfs` driver. Having two different cgroup managers (`cgroupfs` and `systemd`) active simultaneously on the same host causes "cgroup split-brain": resources get misallocated, metrics drift, and under memory pressure `kubelet` will destabilize and terminate pods unpredictably.

- **The Kubernetes rule:** Kubernetes v1.22+ strictly expects both the container runtime (containerd) and the node agent (kubelet) to use the **systemd cgroup driver**.

How to Do It

Perform these steps on all 3 nodes (k8s-cp, k8s-worker-1, k8s-worker-2).

1. Generate the default configuration file

Create the config directory and dump containerd's default generated configuration into /etc/containerd/config.toml:

```
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml > /dev/null
```

2. Enable SystemdCgroup = true

Modify the file so that the runc CRI plugin uses the systemd cgroup driver. Run sed to update the flag in-place:

```
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/g' /etc/containerd/config.toml
```

3. Verification

Verify that the flag was modified properly in the configuration file:

```
grep -n "SystemdCgroup" /etc/containerd/config.toml
```

Verification check: The output must show:

```
SystemdCgroup = true
```

User prompt: what is 2.3, why it is used and how to do that?

Response: **Phase 2.3** is **restarting and enabling the containerd systemd service** so that the configuration changes you made in Phase 2.2 take effect immediately and persist across VM reboots.

What Is It and Why Is It Used?

- **What it is:** systemd is the background service manager on Ubuntu. In Phase 2.2, you modified /etc/containerd/config.toml, but a running process does not automatically read disk modifications while it is running in memory.
- **Why it is used:**
 - **Reloading the config:** Restarting the service forces containerd to parse your new config.toml file, activating the SystemdCgroup = true setting and initializing the CRI Unix socket (/run/containerd/containerd.sock).
 - **Service persistence:** Enabling the service ensures that if your OrbStack machine restarts, containerd starts automatically before kubelet attempts to talk to it. Without this, the node would boot into a degraded state where kubelet fails with socket connection errors.

How to Do It

Perform these steps on all 3 nodes (k8s-cp, k8s-worker-1, k8s-worker-2).

1. Restart and enable containerd

```
sudo systemctl restart containerd
sudo systemctl enable containerd
```

2. Verification

Check the runtime status of the service:

```
sudo systemctl status containerd --no-pager
```

Verification check: Look for the line that says:

```
Active: active (running)
```

If the status shows active (running), containerd is successfully initialized with your new configuration and listening for CRI requests.

User prompt: explain 3.1, why it is used and how to do that?

Response: **Phase 3.1** is **adding the official Kubernetes community APT repository and signing key** to your nodes.

What Is It and Why Is It Used?

- **What it is:** Default Ubuntu package repositories do not maintain current, official releases of Kubernetes tools like kubelet, kubeadm, and kubectl. Kubernetes maintains its own package repositories hosted at pkgs.k8s.io.
- **Why it is used:**

- **Authenticity & Integrity:** Adding the official Kubernetes GPG cryptographic key ensures that every package downloaded via apt is verified, untampered with, and signed directly by the Kubernetes release team.
- **Version Isolation:** Modern Kubernetes package hosting publishes dedicated repository channels per minor version (e.g., v1.34). This prevents unintended upgrades during regular system updates and lets you control your upgrade journey to v1.35 intentionally.

How to Do It

Execute these commands on **all three nodes** (k8s-cp, k8s-worker-1, k8s-worker-2).

1. Install prerequisite utilities

Ensure you have tools to fetch keys and handle HTTPS transports:

```
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gnupg
```

2. Download and store the Kubernetes GPG signing key

Fetch the official public key for the v1.34 channel and store it in /etc/apt/keyrings:

```
sudo mkdir -p -m 755 /etc/apt/keyrings
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.34/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrin
sudo chmod 644 /etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

3. Add the Kubernetes v1.34 APT repository

Create the source list entry pointing to the v1.34 release channel:

```
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.34
```

4. Update your package lists

Refresh the index so your package manager discovers the new packages:

```
sudo apt-get update
```

5. Verification

Verify that APT can see the official Kubernetes packages from the newly added repository:

```
apt-cache madison kubeadm
```

Verification check: The output must display available package candidates matching version 1.34.x pointing to https://pkgs.k8s.io/core:/stable:/v1.34/deb/.

User prompt: explain 3.2, why it is used and how to do that?

Response: **Phase 3.2 is installing the core Kubernetes binaries (kubelet, kubeadm, and kubectl) and pinning (holding) their versions.**

What Are These Tools and Why Are They Used?

Each tool has a dedicated role across your nodes:

- **kubelet (The Node Agent):**
 - Runs on every node in the cluster (control plane and workers).
 - Communicates directly with your container runtime (containerd) via CRI to start, stop, and monitor containers.
 - Continuously watches the API server for Pod specifications assigned to its node and ensures those containers are running and healthy.
- **kubeadm (The Bootstrapper & Lifecycle Manager):**
 - Used to initialize the control plane (kubeadm init), generate bootstrap tokens, join worker nodes (kubeadm join), and coordinate zero-downtime cluster upgrades (kubeadm upgrade).
 - Handles the heavy lifting of TLS certificate generation, static Pod manifest creation, and CoreDNS deployment.
- **kubectl (The CLI Client):**
 - The command-line utility used to communicate with the kube-apiserver via REST calls.
 - Used to inspect cluster state, deploy workloads, manage namespaces, and troubleshoot.
- **Why pin (apt-mark hold) the versions?**
 - By default, running a standard system upgrade (sudo apt-get upgrade) will automatically upgrade packages to the latest available versions.

- In Kubernetes, an unexpected out-of-order component upgrade breaks the strict version-skew policy and can corrupt node configurations. Pinning ensures these binaries only change when you intentionally perform an upgrade procedure.

How to Do It

Execute these steps on **all three nodes** (k8s-cp, k8s-worker-1, and k8s-worker-2).

1. Install the packages

Install kubelet, kubeadm, and kubectl using APT:

```
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
```

(APT will automatically pick up the latest patch version within the v1.34 repository channel you added in Phase 3.1.)

2. Lock the versions

Mark the packages as held so automatic package updates do not touch them:

```
sudo apt-mark hold kubelet kubeadm kubectl
```

3. Verification

Verify the installed versions and ensure the hold locks are applied:

```
kubeadm version -o short
kubectl version --client=true
apt-mark showhold
```

Verification check:

1. Both kubeadm and kubectl will print version strings starting with v1.34.x.
2. The output of apt-mark showhold must list:

```
kubeadm
kubectl
kubelet
```

(Note: If you run `systemctl status kubelet` right now, it will show as `crashlooping` or `waiting`. This is expected behavior; kubelet is waiting for kubeadm to generate its initial cluster configuration and TLS certificates in Phase 4).

User prompt: explain 3.3, why it is used and how to do that?

Response: **Phase 3.3** is **enabling the kubelet systemd service** so that it starts automatically at boot.

What Is It and Why Is It Used?

- **What it is:** kubelet is the only primary Kubernetes component that runs directly as a standard systemd service on the host OS. All other control plane components (API server, scheduler, controller manager, etcd) run as containers managed by the kubelet.
- **Why it is used:**
 - **System Recovery:** If a virtual machine reboots or crashes, systemd must automatically bring kubelet back online without manual intervention.
 - **Bootstrap Handshake:** kubeadm expects the kubelet service to be registered and active with systemd so that when kubeadm init drops the static Pod manifests into /etc/kubernetes/manifests/, kubelet immediately picks them up and spins up the control plane containers.

How to Do It

Run this command on **all three nodes** (k8s-cp, k8s-worker-1, k8s-worker-2):

```
sudo systemctl enable --now kubelet
```

Verification and Expected Behavior

Check the status of the service:

```
sudo systemctl status kubelet --no-pager
```

Verification check:

- The output should state `Loaded: ...; enabled;`
- **Important:** You will see the service status as `activating (auto-restart)` or `crash-looping` with errors like:

```
failed to load kubelet config file /var/lib/kubelet/config.yaml: open /var/lib/kubelet/config.yaml: no
```

This is completely normal and expected at this stage. kubelet is waiting for kubeadm to generate /var/lib/kubelet/config.yaml and cluster certificates. As soon as you run kubeadm init (on the control plane) or kubeadm join (on the workers), kubelet will find this file and transition cleanly to active (running).

User prompt: explain 4.1 in VERY detail, why it is used and how to do that? also lets use flannel for simplicity but will update/change to cilium further(or at last) and also keep in mind to install N version of k8s because we will upgrade to N+1 later.

Response: **Phase 4.1 is defining your cluster's IP address allocation strategy and executing kubeadm init on k8s-cp** to bootstrap the control plane.

What Is It and Why Is It Used?

Running kubeadm init transitions a standalone Linux host into a functional Kubernetes control plane. It coordinates several foundational steps:

- **PKI & Certificate Authority Generation:** Generates dedicated Certificate Authorities (CAs) under /etc/kubernetes/pki for the cluster, API server, front-proxy, and etcd. Every component in Kubernetes uses mutual TLS (mTLS) to authenticate.
- **Control Plane Static Pods:** Writes YAML manifests to /etc/kubernetes/manifests/ for kube-apiserver, kube-controller-manager, kube-scheduler, and etcd. The local kubelet monitors this directory and starts these static pods directly using containerd.
- **Kubeconfig Generation:** Generates administrative credentials (admin.conf), controller credentials, and kubelet credentials with administrative cluster-admin privileges.
- **Cluster Add-ons Setup:** Generates RBAC roles, provisions kube-proxy as a DaemonSet, and schedules CoreDNS to provide internal cluster DNS resolution.

Key Parameters Explained

When executing kubeadm init, two flags are essential:

1. --pod-network-cidr

- **Why it matters:** Defines the virtual IPv4 subnet reserved exclusively for Pods across all nodes.
- **Flannel requirement:** Flannel's upstream manifests are hardcoded by default to expect 10.244.0.0/16. If your initialization does not match this CIDR, Flannel will miscalculate subnet allocations per node and crashloop.
- **Migrating to Cilium later:** Cilium is flexible. When you switch to Cilium down the road, it will automatically detect the cluster's configured --pod-network-cidr (10.244.0.0/16) via the kube-apiserver configmap, making the future CNI migration seamless.

2. --apiserver-advertise-address

- **Why it matters:** Specifies the IP address on k8s-cp that the kube-apiserver binds to and advertises to the outside world.
- **OrbStack context:** OrbStack assigns dynamic private IPs to each VM. If not specified, kubeadm might bind to an internal container bridge or localhost, preventing worker nodes (k8s-worker-1, k8s-worker-2) from reaching the control plane.

3. --kubernetes-version

- **Why it matters:** Explicitly locks the control plane to your initial v1.34 release (e.g., v1.34.0 or stable-1.34), ensuring kubeadm pulls the exact control plane container images intended for your starting point rather than jumping ahead.

How to Do It

Run these steps **only on the control plane node (k8s-cp)**.

Step 1: Identify the Control Plane IP Address

Run:

```
ip -4 addr show eth0
```

Look for the inet line (e.g., 192.168.x.x or 10.x.x.x). Note this IP address down.

Step 2: Pre-pull Control Plane Images (Optional but Recommended)

Verify that your container runtime can pull the required control plane images from registry.k8s.io:

```
sudo kubeadm config images pull --kubernetes-version=v1.34.0
```

Verification check: Images for kube-apiserver, kube-controller-manager, kube-scheduler, etcd, kube-proxy, and pause will pull and show complete without errors.

Step 3: Run kubeadm init

Execute the initialization command, substituting <K8S_CP_IP> with the actual IP address found in Step 1:

```
sudo kubeadm init \
  --apiserver-advertise-address=<K8S_CP_IP> \
  --pod-network-cidr=10.244.0.0/16 \
  --kubernetes-version=v1.34.0
```

What happens behind the scenes during the 1–2 minutes it runs:

1. [preflight] checks swap, ports (6443, 2379–2380, 10250), kernel settings, and CRI socket paths.
2. [certs] creates public/private keys under /etc/kubernetes/pki.
3. [kubeconfig] generates configuration files under /etc/kubernetes/.
4. [kubelet-start] writes /var/lib/kubelet/config.yaml; the kubelet systemd service immediately transitions from crashlooping to active.
5. [control-plane] writes static pod manifests into /etc/kubernetes/manifests/.
6. [etcd] starts the local single-node etcd cluster.
7. [wait-control-plane] polls the kube-apiserver until healthy.
8. [addons] provisions kube-proxy and CoreDNS.

Step 4: Save the Join Command Output

At the end of the output, kubeadm will print a message starting with:

Your Kubernetes control-plane has initialized successfully!

Toward the bottom, it provides a command formatted like:

```
kubeadm join <K8S_CP_IP>:6443 --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash>
```

Copy and save this entire command in a notepad. You will need it in Phase 6 to join k8s-worker-1 and k8s-worker-2.

Step 5: Verification

Confirm that static pod manifests were generated:

```
ls -l /etc/kubernetes/manifests/
```

Verification check: The directory should contain four files:

- etcd.yaml
- kube-apiserver.yaml
- kube-controller-manager.yaml
- kube-scheduler.yaml

Next, check that kubelet is now running stably:

```
sudo systemctl status kubelet --no-pager
```

Verification check: The status should display Active: active (running).

User prompt: explain 4.2 in VERY detail, why it is used and how to do that?

Response: **Phase 4.2 is configuring administrative access (kubeconfig) on the control plane node and bridging it to your host Mac.**

What Is It and Why Is It Used?

When kubeadm init finishes running, it outputs a raw administrative credentials file at /etc/kubernetes/admin.conf.

By default:

- The file is owned strictly by root:root with read permissions restricted to the root user.
- If you run kubectl as your standard non-root user (macbeee), kubectl defaults to checking ~/.kube/config. Finding nothing, it tries connecting to localhost:8080 without TLS and fails immediately with The connection to the server localhost:8080 was refused.
- You do not want to run all cluster operations as sudo kubectl, nor do you want to keep SSH sessions open inside k8s-cp just to deploy manifests.

Setting up Phase 4.2 accomplishes two things:

1. **Local Node Access:** Allows your non-root user on k8s-cp to run kubectl seamlessly.

2. **Mac Host Administration:** Allows your native macOS terminal, shell scripts, and GUI tools (Lens, k9s, VS Code) to authenticate directly to the VM cluster.

Deep Dive: What Lives Inside `admin.conf`?

If you inspect the file, it contains three primary sections:

1. **clusters:** Defines the API server endpoint (e.g., `https://<K8S_CP_IP>:6443`) and embeds the base64-encoded Certificate Authority public key (`certificate-authority-data`) used to encrypt and verify the API server's TLS certificate.
2. **users:** Defines the client credentials for user `kubernetes-admin`, containing base64-encoded client certificate (`client-certificate-data`) and private key (`client-key-data`) signed directly by the cluster's CA. The certificate subject binds this identity to the `system:masters` group, which bypasses all RBAC restrictions and gives superuser powers.
3. **contexts:** Associates the user (`kubernetes-admin`) with the cluster (`kubernetes`) under an active context profile.

How to Do It

Step 1: Configure Access on `k8s-cp` (for non-root user)

Run these commands inside the `k8s-cp` terminal:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Verification check: Run:

```
kubectl get nodes
```

The command should succeed and return:

NAME	STATUS	ROLES	AGE	VERSION
k8s-cp	NotReady	control-plane	2m	v1.34.0

(The status `NotReady` is completely expected right now because the CNI network plugin has not been deployed yet).

Step 2: Bridge Access to your macOS Terminal

Because OrbStack machines share networking directly with macOS, your Mac can communicate with the `k8s-cp` IP address directly.

Open a **new terminal tab on your Mac** (not inside the VM):

1. Create the local `.kube` directory on macOS:

```
mkdir -p ~/.kube
```

2. Copy the config file from OrbStack VM to your Mac: OrbStack provides direct CLI copy and filesystem integration:

```
orb -m k8s-cp cat /etc/kubernetes/admin.conf > ~/.kube/config
```

(If you already have existing clusters in `~/.kube/config`, save it to a separate file like `~/.kube/orb-k8s.yaml` and export `KUBECONFIG=~/.kube/orb-k8s.yaml`).

3. Set safe permissions on your Mac:

```
chmod 600 ~/.kube/config
```

4. Verification from your Mac: Run directly in your Mac terminal:

```
kubectl cluster-info
kubectl get nodes
```

Verification check: The output must show that the Kubernetes control plane is running at `https://<K8S_CP_IP>:6443` and list `k8s-cp` without needing `ssh` or `orb` prefixes.

To understand how contexts, certificates, and cluster definitions bind together inside this configuration file, check out [kubeconfig File Explained](#). This breakdown clearly visualizes the interaction between `kubectl`, authentication certificates, and API endpoints.

```
User prompt: macbeee@k8s-cp:/Users/macbeee$ mkdir -p $HOME/.kube macbeee@k8s-cp:/Users/macbeee$ sudo cp -i
/etc/kubernetes/admin.conf $HOME/.kube/config macbeee@k8s-cp:/Users/macbeee$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
macbeee@k8s-cp:/Users/macbeee$ kubectl get nodes NAME STATUS ROLES AGE VERSION k8s-cp NotReady control-plane
94m v1.34.11 macbeee@k8s-cp:/Users/macbeee$ exit logout macbeee@macbees-mac ~ % mkdir -p ~/.kube macbeee@macbees-mac ~ %
orb -m k8s-cp cat /etc/kubernetes/admin.conf > ~/.kube/config macbeee@macbees-mac ~ % cat ~/.kube/config cat: /etc/kubernetes/admin.conf:
Permission denied macbeee@macbees-mac ~ % sudo cat ~/.kube/config Password: cat: /etc/kubernetes/admin.conf: Permission denied
macbeee@macbees-mac ~ % sudo cat ~/.kube/config cat: /etc/kubernetes/admin.conf: Permission denied macbeee@macbees-mac ~ % sudo
cat ~/.kube/config cat: /etc/kubernetes/admin.conf: Permission denied macbeee@macbees-mac ~ % ll ~/.kube zsh: command not found: ll
macbeee@macbees-mac ~ % ls -ltr ~/.kube total 8 -rw-r--r-- 1 macbeee staff 52 13 Sep 00:43 config macbeee@macbees-mac ~ % sudo view
~/.kube/config macbeee@macbees-mac ~ % chmod 600 ~/.kube/config macbeee@macbees-mac ~ % kubectl cluster-info error: error loading
config file "/Users/macbeee/.kube/config": yaml: mapping values are not allowed in this context
```

Response: The issue happened because `/etc/kubernetes/admin.conf` inside the VM is owned by root with `0600` permissions.

When you ran `orb -m k8s-cp cat /etc/kubernetes/admin.conf`, it ran as standard user `macbeee` inside the VM, hit `cat: /etc/kubernetes/admin.conf: Permission denied`, and redirected that exact plain-text error string into your Mac's `~/kube/config`. That is why `kubectl` failed parsing invalid YAML.

Run these steps from your **Mac terminal** to fix it:

Step 1: Copy from the VM User Config

Since you already copied the file to your VM user's home directory (`$HOME/~/kube/config`) and gave `macbeee` ownership in the previous step, read from that file instead:

```
orb -m k8s-cp cat ~/kube/config > ~/kube/config
chmod 600 ~/kube/config
```

(Alternatively, if reading directly from `/etc`, you would need `sudo`: `orb -m k8s-cp sudo cat /etc/kubernetes/admin.conf > ~/kube/config`).

Step 2: Verification

Inspect the contents of the file on your Mac:

```
head -n 5 ~/kube/config
```

Verification check: The first line must read `apiVersion: v1` and `kind: Config` (valid YAML, not a "Permission denied" error string).

Then test the connection from your Mac:

```
kubectl cluster-info
kubectl get nodes
```

Verification check: `kubectl cluster-info` will return `Kubernetes control plane is running at https://...`, and `kubectl get nodes` will list `k8s-cp` with status `NotReady`.

User prompt: explain 4.3 (Deploys the cluster-wide kube-proxy DaemonSet and CoreDNS ConfigMap/Deployment.), why it is used and how to do that?

Response: **Phase 4.3** covers the two essential in-cluster networking add-ons that `kubeadm init` generates during bootstrap: the **kube-proxy DaemonSet** and the **CoreDNS Deployment**.

What Are They and Why Are They Used?

1. kube-proxy (Service Networking Engine)

- **What it is:** A network proxy running as a `DaemonSet` (one Pod per node across both control plane and workers).
- **Why it is used:**
 - In Kubernetes, Pod IPs are ephemeral and frequently change when Pods recreate. A `Service` gives a static virtual IP (`ClusterIP`) to front a group of Pods.
 - `kube-proxy` watches the Kubernetes API server for additions and removals of `Service` and `Endpoints / EndpointSlice` objects.
 - It programs the host Linux networking subsystem (by default using `iptables`, or optionally `IPVS`) so that any packet sent to a `ClusterIP` or `NodePort` is intercepted and translated (DNAT) to one of the backend Pod IPs.
 - Without `kube-proxy`, Kubernetes `Services` do not function—Pods cannot talk to other Pods via `Services`, and external traffic hitting `NodePort` fails to reach your app.

2. CoreDNS (Cluster Internal DNS)

- **What it is:** A flexible, extensible DNS server deployed as a standard Kubernetes `Deployment` (defaulting to 2 replicas) along with a `ConfigMap` (`coredns`).
- **Why it is used:**
 - Provides service discovery across the entire cluster.
 - Instead of needing to know a service's dynamic IP address, applications query names like `my-service.default.svc.cluster.local` or simply `http://my-service`.
 - `CoreDNS` queries the Kubernetes API server to map service names directly to their active `ClusterIP`.
 - Without `CoreDNS`, pods cannot resolve domain names inside the cluster, breaking inter-service communication.

How to Do It

kubeadm init handles generating and applying the manifests for both kube-proxy and CoreDNS automatically during Phase 4.1. **You do not need to install them manually.**

However, you must inspect their state, understand their current lifecycle condition, and verify their configuration.

Step 1: Inspect the Generated Add-ons

Run these commands directly from your Mac terminal (or from k8s-cp):

1. Check the DaemonSet and Deployment objects:

```
kubectl get daemonset -n kube-system
kubectl get deployment -n kube-system
```

Verification check:

- You will see kube-proxy listed as a DaemonSet with desired/current replicas matching your node count.
- You will see coredns listed as a Deployment with 2 desired replicas.

2. Check the running pods:

```
kubectl get pods -n kube-system -o wide
```

What you will observe:

- kube-proxy-... will show Running on k8s-cp.
- coredns-... pods will be in a Pending state.

Step 2: Why Is CoreDNS in Pending? (The CNI Dependency)

CoreDNS pods will remain in Pending with the event 0/1 nodes are available: 1 node(s) had untolerated taint {node.kubernetes.io/not-ready: }.

- Why: CoreDNS pods require an IP address from the Pod CIDR (10.244.0.0/16) to run.
- A node cannot assign IP addresses to Pods until a **Container Network Interface (CNI)** plugin—such as Flannel or Calico—is installed.
- As soon as you deploy the CNI in **Phase 5**, the CNI configures the network bridge, clears the node.kubernetes.io/not-ready taint, and CoreDNS immediately schedules and transitions to Running.

Step 3: Inspect the CoreDNS ConfigMap (Optional Knowledge)

If you want to see how CoreDNS translates domain names inside the cluster, view its ConfigMap:

```
kubectl get configmap coredns -n kube-system -o yaml
```

You will see the Corefile directive configuring the .cluster.local domain, upstream forwarding to /etc/resolv.conf, and Prometheus metrics endpoints.

```
User prompt: macbeee@macbees-mac ~ % kubectl get daemonset -n kube-system NAME DESIRED CURRENT READY UP-TO-
DATE AVAILABLE NODE SELECTOR AGE kube-proxy 1 1 1 1 1 1 kubernetes.io/os=linux 108m
macbeee@macbees-mac ~ % kubectl get deployment -n kube-system NAME READY UP-TO-DATE AVAILABLE AGE coredns 0/2 2
0 108m macbeee@macbees-mac ~ % kubectl get pods -n kube-system -o wide NAME READY STATUS
RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES coredns-66bc5c9577-88jlm 0/1 Pending 0
108m <none> <none> <none> <none> coredns-66bc5c9577-btvwf 0/1 Pending 0 108m <none> <none>
<none> <none> etcd-k8s-cp 1/1 Running 0 109m 192.168.139.197 k8s-cp <none> <none> kube-
apiserver-k8s-cp 1/1 Running 0 109m 192.168.139.197 k8s-cp <none> <none> kube-controller-manager-k8s-cp 1/1
Running 0 109m 192.168.139.197 k8s-cp <none> <none> kube-proxy-wmmjw 1/1 Running 0 108m
192.168.139.197 k8s-cp <none> <none> kube-scheduler-k8s-cp 1/1 Running 0 109m 192.168.139.197 k8s-cp
<none> <none> macbeee@macbees-mac ~ % kubectl get configmap coredns -n kube-system -o yaml apiVersion: v1 data: Corefile: |
..:53 { errors health { lameduck 5s } ready kubernetes cluster.local in-addr.arpa ip6.arpa { pods insecure
fallthrough in-addr.arpa ip6.arpa ttl 30 } prometheus :9153 forward . /etc/resolv.conf { max_concurrent 1000
} cache 30 { disable success cluster.local disable denial cluster.local } loop reload loadbalance }
kind: ConfigMap metadata: creationTimestamp: "2026-09-12T17:37:43Z" name: coredns namespace: kube-system resourceVersion: "222"
uid: 0bd32ce2-0706-4c5d-9499-83937015c046 macbeee@macbees-mac ~ % The above is just for the context, dont generate anything.
```

Response: Understood. Standing by for your next instruction.

User prompt: there was some typo above. explain 4.3 (Configure Access (kubeconfig): Copy admin.conf to your local non-root user directory on k8s-cp. Copy this file to your macOS host (~/.kube/config) so your local terminal has administrative cluster access.), why it is used and how to do that?

Response: **Phase 4.3 is configuring client credentials (kubeconfig) so non-root environments—both within the VM and on your host Mac—can securely communicate with the Kubernetes API server.**

Why It Is Used

When `kubeadm init` bootstraps the control plane, it writes the cluster's administrative credentials to `/etc/kubernetes/admin.conf`.

1. Permission Boundaries on `k8s-cp`:

- `/etc/kubernetes/admin.conf` has file permissions set to `0600` and is owned strictly by `root:root`.
- Running daily operational commands as `root` (or prefixing everything with `sudo kubectl`) breaks auditing, risks accidental OS modifications, and conflicts with user-level environment tools.
- `kubectl` looks by default for a configuration file located at `$HOME/.kube/config`. Copying `admin.conf` into your user's home directory and reassigning ownership (`chown`) allows your regular user (`macbeee`) to execute `kubectl` commands without `sudo`.

2. Decoupling Host Administration (macOS Access):

- Running `kubectl` inside an SSH or VM shell session adds friction.
- Kubernetes uses an HTTP/REST API over TLS (port 6443). Because OrbStack automatically maps the VM's network interface directly to macOS, your Mac can route traffic to `[https://192.168.139.197:6443]` (`https://192.168.139.197:6443`) just like any external client on the network.
- Placing these credentials into your Mac's `~/.kube/config` lets you use native macOS terminals, IDEs, Helm, k9s, and CI/CD tools directly against the cluster.

What Lives Inside `admin.conf`?

The file contains three critical sections that establish a secure TLS session:

- **clusters:** Contains the endpoint URL (`[https://192.168.139.197:6443]` (`https://192.168.139.197:6443`)) and the base64-encoded root Certificate Authority (`certificate-authority-data`). Your client uses this CA to verify the API server's identity and prevent man-in-the-middle attacks.
- **users:** Contains the identity `kubernetes-admin`, paired with a client certificate (`client-certificate-data`) and a private key (`client-key-data`). This certificate is signed by the cluster CA and belongs to the `system:masters` group, granting full cluster-admin privileges.
- **contexts:** Defines the pairing between the `kubernetes-admin` user and the `kubernetes` cluster as the active context.

How to Do It

Step 1: Set Up Non-Root Access on `k8s-cp`

Run this inside the `k8s-cp` VM:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

- **`mkdir -p $HOME/.kube`:** Creates the hidden directory where `kubectl` expects its default config.
- **`sudo cp -i ...`:** Copies the root-only file into the user directory.
- **`sudo chown $(id -u):$(id -g) ...`:** Changes the file ownership from `root:root` to your user ID and primary group ID (`macbeee:macbeee`), enabling non-root reads and writes.

Verification on `k8s-cp`:

```
kubectl get nodes
```

Verification check: The command executes without `sudo` and returns `k8s-cp` with status `NotReady` (expected until CNI is applied).

Step 2: Export Access to Your macOS Host

Run these commands directly from your **Mac terminal** (not inside the VM):

1. Create the `.kube` directory on macOS:

```
mkdir -p ~/.kube
```

2. Copy the configuration file from the VM to macOS: Since `macbeee` now owns `$HOME/.kube/config` inside `k8s-cp`, you can extract it cleanly without permission errors:

```
orb -m k8s-cp cat ~/.kube/config > ~/.kube/config
```

(Note: If you ever read directly from `/etc/kubernetes/admin.conf`, you must prepend `sudo` so the command has permission to read the root-owned source: `orb -m k8s-cp sudo cat /etc/kubernetes/admin.conf > ~/.kube/config`.)

3. Restrict permissions on macOS: Ensure only your Mac user can read the sensitive private key:

```
chmod 600 ~/.kube/config
```

Verification from macOS

Verify that `kubectl` on your Mac communicates with the VM API server:

```
kubectl cluster-info
```

Verification check: The output must confirm:

Kubernetes control plane is running at `https://192.168.139.197:6443`

Check the registered nodes:

```
kubectl get nodes
```

Verification check: The node list displays `k8s-cp` running version `v1.34.x` directly from your Mac terminal.

User prompt: explain 5.1 in VERY detail, why it is used and how to do that?

Response: **Phase 5.1 is selecting, evaluating, and preparing the Container Network Interface (CNI) plugin** for your cluster.

What Is It and Why Is It Used?

Out of the box, Kubernetes deliberately ships **without** an active network implementation for Pods.

1. The Fundamental Kubernetes Networking Model

Kubernetes enforces strict networking requirements across any conformant cluster:

- Every Pod gets its own unique, routable IP address.
- Pods on any node can communicate with Pods on every other node **without using NAT** (Network Address Translation).
- Agents on a node (like `kubelet`) can communicate with all Pods on that same node.
- A Pod sees itself with the exact same IP address that every other Pod sees it as.

2. Why the Cluster Cannot Function Without a CNI

Until a CNI is deployed:

- **The Node stays NotReady:** The `kubelet` scans the host directory `/etc/cni/net.d/` for a valid network configuration. Finding none, it reports `NetworkPluginNotReady` and taints the node with `node.kubernetes.io/not-ready:NoSchedule`.
- **CoreDNS stays in Pending:** CoreDNS pods cannot bind to the host network; they require dynamic IP addresses assigned out of your `pod-network-cidr` (`10.244.0.0/16`). Because the node has no IP allocation mechanism, the scheduler cannot place them.
- **No Virtual Interfaces:** There is no bridge (like `cni0`) or virtual ethernet pairs (`veth`) created to pipe container traffic out to the host network.

3. How a CNI Operates Under the Hood

When `kubelet` needs to spin up a container via `containerd`:

1. `containerd` creates a Linux network namespace for the new container.
2. It executes the CNI binary on the host passing the action (ADD), the namespace path, and the container ID.
3. The CNI binary creates a `veth` pair: one end moves inside the container's network namespace (becoming `eth0`), and the other end attaches to a virtual bridge (e.g., `cni0`) on the host.
4. The CNI's IPAM (IP Address Management) plugin claims an unused IP from the node's allocated subnet slice and assigns it to the container's interface.
5. Routing rules and encapsulation tunnels (like VXLAN) are written to ensure packets leave the VM and land safely inside the target Pod on another VM.

Comparison: Flannel vs. Calico vs. Cilium

Before deploying, understanding why Flannel makes sense now—and how Cilium compares—ensures you know what is happening under the hood:

Feature	Flannel (Your Current Step)	Calico	Cilium (Your Future Goal)
Primary Goal	Minimal, dead-simple L3 overlay network	Enterprise networking & granular security	High-performance, eBPF-powered routing & observability
Data Plane	Linux kernel VXLAN / Host-GW	Linux iptables / IP routing / eBPF	Linux eBPF (bypasses iptables entirely)
Kubernetes NetworkPolicy	No (Traffic filtering not supported)	Yes (Rich Layer 3/4 policies)	Yes (Layer 3/4 + Layer 7 API-aware policies)
Resource Overhead	Extremely low (~20–30 MB RAM per node)	Moderate	Higher (requires compiling/loading eBPF programs)

Feature	Flannel (Your Current Step)	Calico	Cilium (Your Future Goal)
kube-proxy Dependency	Requires kube-proxy for Service routing	Usually uses kube-proxy	Can completely replace kube-proxy

Starting with **Flannel** keeps your initial setup simple: it provides the standard overlay network without complex custom resource definitions (CRDs), complex BGP peering, or eBPF kernel hooks. This lets you observe the core networking behavior before migrating to advanced tooling later.

How to Prepare and Inspect Before Applying

You do not run this inside the VM; you can inspect and apply it directly from your **Mac terminal** using your configured `kubectl`.

Step 1: Examine the Official Flannel Manifest

Flannel publishes its deployment manifest directly from its GitHub repository:

```
curl -sSL https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml | grep -A 5 -B 2
```

Verification check: Look at the embedded ConfigMap section in the manifest:

```
net-conf.json: |
{
  "Network": "10.244.0.0/16",
  "EnableNFTables": false,
  "Backend": {
    "Type": "vxlan"
  }
}
```

Notice `"Network": "10.244.0.0/16"`. This matches the exact `--pod-network-cidr=10.244.0.0/16` flag you passed during `kubeadm init` in Phase 4.1. Because they match, Flannel knows precisely which IP block to slice up per node.

Step 2: Understand the Subnet Allocation Strategy

With `10.244.0.0/16`:

- Flannel assigns each registered node its own /24 subnet slice (providing 254 usable IPs per node).
- Example allocation:
 - `k8s-cp: 10.244.0.0/24` (Pods get `10.244.0.1` to `10.244.0.254`)
 - `k8s-worker-1: 10.244.1.0/24`
 - `k8s-worker-2: 10.244.2.0/24`
- When a Pod on `k8s-worker-1` (`10.244.1.5`) pings a Pod on `k8s-worker-2` (`10.244.2.9`), the Flannel daemon intercepts the packet, encapsulates it in a standard UDP VXLAN packet (port 8472), routes it across the physical network via the VM IP (`192.168.x.x`), and unpacks it on the destination node.

User prompt: do i have to run this: 'curl -sSL https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml | grep -A 5 -B 2 'net-conf.json'' in mac's terminal or k8s-cp terminal?

Response: You can run it in **either terminal**—it does not change any state or configure anything on the machine.

That command is purely a read-only inspection tool:

- `curl` downloads the text of the YAML file from GitHub into memory.
- `grep` filters the text so you can confirm that Flannel's configured subnet matches your `--pod-network-cidr=10.244.0.0/16`.

Since your Mac terminal has direct internet access and already has `kubectl` configured, running it directly in your **Mac terminal** is the most convenient choice.

User prompt: is 5.1 done? if yes, then explain 5.2 in VERY detail, why it is used and how to implement that?

Response: Yes, **Phase 5.1** is done. You verified the networking concept, confirmed that Flannel uses the matching `10.244.0.0/16` CIDR, and prepared for deployment.

Phase 5.2: Deploying Flannel CNI & Activating the Control Plane

Phase 5.2 is applying the Flannel CNI manifests to your cluster and verifying the network interface bootstrap.

Why It Is Used

Right now, your control plane node is blocked from scheduling workloads:

- `k8s-cp` has the status `NotReady` because `kubelet` detects no active CNI configuration in `/etc/cni/net.d/`.
- `CoreDNS` pods are stuck in `Pending` because they have no network interfaces or IP addresses to attach to.

Applying Flannel resolves this by introducing the networking foundation:

1. Host Configuration Injection (Init Container):

- Flannel runs an `initContainer` on the host that writes `/etc/cni/net.d/10-flannel.conflist`.
- It unpacks the standard CNI network plugins (`bridge`, `host-local`, `portmap`) into `/opt/cni/bin/`.
- As soon as `kubelet` detects valid CNI configuration files in `/etc/cni/net.d/`, it drops the `NetworkPluginNotReady` taint.

2. Subnet Leasing via Kubernetes API:

- The Flannel daemon queries the Kubernetes API server and annotates the `k8s-cp` node with its dedicated subnet slice (e.g., `[flannel.alpha.coreos.com/pod-cidr] (https://flannel.alpha.coreos.com/pod-cidr): 10.244.0.0/24`).

3. Bridge Creation (`cni0` & `flannel.1`):

- Creates a virtual bridge device (`cni0`) on the Linux OS.
- Creates a VXLAN overlay tunnel device (`flannel.1`) listening on UDP port 8472 for cross-node pod routing.

4. Triggering CoreDNS Scheduling:

- With the node reporting `Ready`, the Kubernetes scheduler immediately places the pending `CoreDNS` pods onto `k8s-cp`, assigns them IP addresses from `10.244.0.0/24`, and boots them into `Running` status.

What Gets Created Inside the Manifest

When you apply `kube-flannel.yml`, Kubernetes provisions:

- **Namespace / ServiceAccount:** `kube-flannel` service account in the `kube-flannel` namespace.
- **ClusterRole & ClusterRoleBinding:** Permissions allowing Flannel to read node details, modify node annotations, and monitor Pod network requirements.
- **ConfigMap (`kube-flannel-cfg`):** Contains the CNI network specification (`cni-conf.json`) and the VXLAN backend definition (`net-conf.json`).
- **DaemonSet (`kube-flannel-ds`):** Ensures one Flannel agent pod runs on every existing and future node in your cluster.

How to Implement It

Run these commands directly from your **Mac terminal** (where your working `kubectl` is configured):

Step 1: Apply the Manifest

Apply the official Flannel manifest:

```
kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
```

Step 2: Monitor Deployment Progression

Watch the Flannel pods and CoreDNS pods spin up:

```
kubectl get pods -n kube-flannel -w
```

(Press `Ctrl + C` once the Flannel pod shows `Running` with 1/1 containers ready).

Then check the CoreDNS pods in `kube-system`:

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
```

Verification check: Both `coredns` pods must transition from `Pending` to **Running**.

Step 3: Verify Node Health

Check your node status:

```
kubectl get nodes -o wide
```

Verification check: `k8s-cp` must now show **STATUS: Ready**.

Step 4: Verify Host Interfaces and Node Subnet (Deep Dive Verification)

To see the direct low-level effects of Phase 5.2, run these commands inside the `k8s-cp` VM:

1. Inspect the allocated subnet annotation:

```
kubectl get node k8s-cp -o jsonpath='{.metadata.annotations.flannel.alpha.coreos.com/pod-cidr}'
echo ""
```

Verification check: Output will display something like 10.244.0.0/24.

2. Check the generated host interfaces: Run inside k8s-cp:

```
orb -m k8s-cp ip -brief address show
```

Verification check: You will now see two new virtual network devices created on the machine:

- **flannel.1:** The VXLAN tunnel endpoint.
- **cni0:** The software bridge connecting the pods.

```
User prompt: macbeee@k8s-cp:/Users/macbeee$ kubectl get node k8s-cp -o
jsonpath='{.metadata.annotations.flannel.alpha.coreos.com/pod-cidr}' echo "" macbeee@k8s-cp:/Users/macbeee$ kubectl get node k8s-cp -o
jsonpath='{.metadata.annotations.flannel.alpha.coreos.com/pod-cidr}' macbeee@k8s-cp:/Users/macbeee$ echo "" macbeee@k8s-
cp:/Users/macbeee$ orb -m k8s-cp ip -brief address show lo UNKNOWN 127.0.0.1/8 ::1/128 eth0@if9 UP
192.168.139.197/24 metric 100 fd07:b51a:cc66:0:f0aa:d9ff:fee8:ed27/64 fe80::f0aa:d9ff:fee8:ed27/64 flannel.1 UNKNOWN
10.244.0.0/32 fe80::e84d:95ff:fe17:f176/64 cni0 UP 10.244.0.1/24 fe80::d827:bcff:feba:3cd5/64 veth78eb94f1@if2 UP
fe80::b472:e8ff:fed7:7082/64 veth1ca6b2fd@if2 UP fe80::387f:70ff:fe65:f39/64 macbeee@k8s-cp:/Users/macbeee$
```

Response: Your CNI deployment is fully active and functioning as expected.

Notice what your `ip -brief address show` output reveals:

- **flannel.1** has claimed 10.244.0.0/32 (the VXLAN overlay endpoint).
- **cni0** is assigned 10.244.0.1/24 (the bridge gateway for Pods scheduled on k8s-cp).
- **veth...** pairs have been created and bound to the bridge. These are the two virtual ethernet pairs plugging directly into the two CoreDNS pods.

(Note: The annotation command was blank because modern Flannel uses the native `spec.podCIDR` field assigned by kubeadm instead of an annotation. You can see it with `kubectl get node k8s-cp -o jsonpath='{.spec.podCIDR}'`, which will print 10.244.0.0/24.)

Check your node status now:

```
kubectl get nodes
```

Verification check: k8s-cp will now show **STATUS: Ready**.

Phase 6: Joining Worker Nodes (k8s-worker-1 & k8s-worker-2)

Now that the control plane is fully operational, you are ready to bootstrap and join the two worker nodes.

What Happens During kubeadm join?

When you run the join command on a worker:

1. **Discovery & Validation:** The worker contacts the API server at 192.168.139.197:6443 using the bootstrap token. It downloads the cluster CA and verifies the CA's public key fingerprint against `--discovery-token-ca-cert-hash`.
2. **TLS Bootstrapping:** The worker generates a local private key and submits a Certificate Signing Request (CSR) to the API server to obtain a client certificate for its own kubelet identity (`system:node:<node-name>`).
3. **Kubelet Configuration:** kubeadm generates `/var/lib/kubelet/config.yaml` and starts managing the local kubelet.
4. **DaemonSet Workloads Inbound:** As soon as the API server registers the new node:
 - The kube-proxy DaemonSet schedules a proxy pod on the worker.
 - The kube-flannel DaemonSet schedules a flannel pod on the worker, which sets up `cni0` and `flannel.1` with a new slice (e.g., 10.244.1.0/24).
 - The worker transitions to Ready.

Step 1: Ensure Prerequisites Are Completed on Worker Nodes

Before joining, verify that **Phase 1, 2, and 3** were completed on both k8s-worker-1 and k8s-worker-2:

- Kernel modules (overlay, br_netfilter) loaded and sysctl rules applied.
- containerd installed with `SystemdCgroup = true` and running.
- kubelet, kubeadm, and kubectl installed (v1.34.x) and pinned with `apt-mark hold`.

Step 2: Retrieve the Join Command

If you saved the join command from Phase 4.1, you can use it directly.

If you lost it or the 24-hour token expired, generate a fresh join command on `k8s-cp` (or from your Mac terminal):

```
kubeadm token create --print-join-command
```

It will print a command in this exact format:

```
sudo kubeadm join 192.168.139.197:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash>
```

Step 3: Execute the Join Command on Both Workers

1. Shell into `k8s-worker-1` and run the join command with `sudo`:

```
sudo kubeadm join 192.168.139.197:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash>
```

2. Shell into `k8s-worker-2` and run the exact same join command with `sudo`:

```
sudo kubeadm join 192.168.139.197:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash>
```

Verification check on each worker: The output should conclude with:

This node has joined the cluster:

```
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.
```

Run `'kubectl get nodes'` on the control-plane to see this node join the cluster.

Step 4: Verification from macOS

From your Mac terminal, watch all 3 nodes report and turn ready:

```
kubectl get nodes -o wide
```

Verification check: All three nodes (`k8s-cp`, `k8s-worker-1`, `k8s-worker-2`) must display **STATUS: Ready** with version `v1.34.x`.

Next, verify that `kube-proxy` and `kube-flannel` are actively running across all 3 nodes:

```
kubectl get pods -n kube-system -l k8s-app=kube-proxy -o wide
kubectl get pods -n kube-flannel -o wide
```

Verification check: Both outputs should show 3 separate Pods running—exactly one per node.

User prompt: is phase 6 completed? just reply yes or no.

Response: No.

User prompt: okay what is remained? explain it in VERY detail, why it is used and how to implement that?

Response: What remains to complete **Phase 6** is **actually running the bootstrap join procedure on both worker nodes (`k8s-worker-1` and `k8s-worker-2`) and validating that they successfully attach to your control plane.**

What Is This Step and Why Is It Used?

Until you join these nodes, you only have a single-node control plane. A production-like Kubernetes architecture separates control workloads from tenant workloads:

- **Separation of Concerns:** The control plane (`k8s-cp`) is tainted by default (`node-role.kubernetes.io/control-plane:NoSchedule`) so general application Pods do not consume CPU or memory needed by `etcd` or the API server.
- **Workload Distribution:** `k8s-worker-1` and `k8s-worker-2` provide the compute, memory, and networking capacity where your test applications, databases, and CSI drivers will actually run.
- **The Bootstrap Process:** Running `kubeadm join` initiates a secure cryptographic handshake. The worker node uses a short-lived token to discover the cluster, downloads and validates the cluster Certificate Authority (CA) public key, submits a Certificate Signing Request (CSR) to the API server to obtain its own node TLS certificates, and configures the local `kubelet` `systemd` daemon to register the worker with the cluster.

How to Implement It

Step 1: Complete Phase 1, 2, and 3 on Both Worker VMs

If you have not already prepared `k8s-worker-1` and `k8s-worker-2`, they need the exact same foundational OS and container runtime configuration you did on `k8s-cp`:

1. **Disable swap:** Ensure `free -m` reports `0` for swap.
2. **Kernel modules:** Load `overlay` and `br_netfilter`.

3. **Sysctl network parameters:** Set `net.bridge.bridge-nf-call-iptables = 1`, `net.bridge.bridge-nf-call-ip6tables = 1`, and `net.ipv4.ip_forward = 1` via `sudo sysctl --system`.
4. **Containerd:** Install `containerd.io`, generate the default config to `/etc/containerd/config.toml`, set `SystemdCgroup = true`, then enable and start the service.
5. **Kubernetes packages:** Add the v1.34 APT repository, install `kubelet`, `kubeadm`, and `kubectrl`, hold their versions with `sudo apt-mark hold kubelet kubeadm kubectrl`, and enable the service with `sudo systemctl enable --now kubelet`.

Step 2: Retrieve / Generate the Join Command

From your **Mac terminal**, generate a fresh join command with a new token and the correct CA cert hash:

```
orb -m k8s-cp sudo kubeadm token create --print-join-command
```

You will receive an output in this exact structure:

```
sudo kubeadm join 192.168.139.197:6443 --token <token_string> --discovery-token-ca-cert-hash sha256:<hash_
```

Step 3: Execute the Join Command on k8s-worker-1

Open a shell into `k8s-worker-1` (or run it via `orb` from your Mac):

```
orb -m k8s-worker-1
```

Run the join command with `sudo`:

```
sudo kubeadm join 192.168.139.197:6443 --token <token_string> --discovery-token-ca-cert-hash sha256:<hash_
```

What happens behind the scenes:

- `[preflight]` verifies kernel parameters, ports, and runtime sockets.
- `[discovery]` reaches `[https://192.168.139.197:6443]` (`https://192.168.139.197:6443`), validates the cluster identity using the CA hash, and pulls cluster metadata.
- `[kubelet-start]` generates `/var/lib/kubelet/bootstrap-kubeconfig` and `/var/lib/kubelet/config.yaml`.
- The `kubelet` starts up, registers itself as `k8s-worker-1` with the API server, and receives an approved client certificate.

Step 4: Execute the Join Command on k8s-worker-2

Open a shell into `k8s-worker-2` (or run it via `orb` from your Mac):

```
orb -m k8s-worker-2
```

Run the exact same join command with `sudo`:

```
sudo kubeadm join 192.168.139.197:6443 --token <token_string> --discovery-token-ca-cert-hash sha256:<hash_
```

Step 5: End-of-Phase Verification

Once both nodes finish joining, run these commands directly from your **Mac terminal**:

1. Check Node Readiness:

```
kubectrl get nodes -o wide
```

Verification check:

- You must see three rows: `k8s-cp`, `k8s-worker-1`, and `k8s-worker-2`.
- All three nodes must show **STATUS: Ready**.
- The version on all three must be identical (`v1.34.x`).

2. Verify DaemonSet Expansion: Because `kube-proxy` and `kube-flannel` are DaemonSets, Kubernetes will automatically schedule one Pod on each new worker node:

```
kubectrl get pods -n kube-system -l k8s-app=kube-proxy -o wide
kubectrl get pods -n kube-flannel -o wide
```

Verification check:

- Both commands must list **3 running pods** each—one running on `k8s-cp`, one on `k8s-worker-1`, and one on `k8s-worker-2`.
- Each node's Flannel pod automatically provisions the local `cnio` bridge and VXLAN routing table on that machine.

```
User prompt: macbeee@macbees-mac ~ % kubectrl get pods -n kube-system -l k8s-app=kube-proxy -o wide
NAME                                READY STATUS
RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
192.168.139.142 k8s-worker-1 <none> <none> kube-proxy-5bw24 1/1 Running 0 70s 192.168.139.169 k8s-worker-2
```

```

<none>      <none> kube-proxy-wmmjw 1/1 Running 0 177m 192.168.139.197 k8s-cp <none> <none>
macbeeee@macbees-mac ~ % kubectl get pods -n kube-flannel -o wide NAME READY STATUS RESTARTS AGE IP
NODE NOMINATED NODE READINESS GATES kube-flannel-ds-2mtz4 1/1 Running 0 2m7s 192.168.139.169 k8s-
worker-2 <none> <none> kube-flannel-ds-4jtpl 1/1 Running 0 3m1s 192.168.139.142 k8s-worker-1 <none> <none>
kube-flannel-ds-xg4k4 1/1 Running 0 26m 192.168.139.197 k8s-cp <none> <none> macbeeee@macbees-mac ~ % is
Phase 6 done now? just reply yes or no.

```

Response: Yes.

User prompt: now, first explain me phase 7 in VERY detail and why it is used then explain me 7.1 in VERY detail, why it is used and how to do that?

Response: Deep Dive: Phase 7 (Dynamic Storage & CSI Implementation)

What Is Phase 7 and Why Is It Used?

Up to this point, your cluster can schedule pods and route network traffic, but all storage inside your containers is **ephemeral**. If a pod crashes or gets rescheduled onto another worker node, all data written inside its container filesystem is permanently lost.

To run databases (PostgreSQL, MySQL), stateful applications, or persistent caches, Kubernetes uses three storage abstractions:

1. **PersistentVolume (PV):** A piece of storage in the cluster provisioned by an administrator or dynamically provisioned by a plugin.
2. **PersistentVolumeClaim (PVC):** A request for storage by a user/pod (e.g., "give me 5Gi of ReadWriteOnce storage").
3. **StorageClass:** Defines the "class" or provisioner of storage and automates the creation of PVs when PVCs are requested.

The Problem: Kubernetes Has No Built-in Local Dynamic Provisioner

In bare-metal, virtualized, or homelab environments without cloud provider APIs (like AWS EBS or GCP Persistent Disk), Kubernetes only provides primitive primitives like `hostPath` or `emptyDir`:

- With raw `hostPath`, an administrator must manually log into each worker node, make a directory (`mkdir /data`), manually create a static `PersistentVolume` YAML mapping to that directory, and bind it.
- This does not scale and breaks automation.

What Is a CSI (Container Storage Interface)?

The **CSI** is an industry-standard specification that allows third-party storage providers to develop storage plugins for Kubernetes without modifying the core Kubernetes codebase.

- A CSI driver listens for `PersistentVolumeClaim` events via the Kubernetes API.
- When a developer asks for a volume, the CSI driver dynamically creates a directory or block device on the assigned node's disk and binds it to the pod on demand.

Phase 7 is implementing dynamic storage via a CSI driver so that whenever any application requests persistent disk space, the cluster automatically carves out and attaches that storage with zero manual intervention.

Phase 7.1: Selecting and Evaluating the CSI Provider

What Is Phase 7.1 and Why Is It Used?

Before installing software, you must evaluate the storage architecture that best fits your environment: a 3-node cluster running inside lightweight OrbStack Linux machines on Apple Silicon macOS.

There are two primary architectural categories of Kubernetes storage:

1. Distributed Replicated Block Storage (e.g., Longhorn, Rook-Ceph)

- **How it works:** Requires raw unformatted block disks attached to each worker node. It replicates every write across the network to multiple nodes for high availability.
- **Why it is impractical here:**
 - Requires substantial CPU and RAM overhead (~1.5–2 GB RAM per node just for storage daemons).
 - Requires dedicated virtual block disks attached to the OrbStack VMs, which adds unnecessary complexity to a local development environment.
 - Excessive disk I/O virtualization overhead on your Mac's SSD.

2. Local Host Dynamic Provisioning (Rancher Local Path Provisioner) — The Target Solution

- **How it works:** A minimal, lightweight CSI-like controller maintained by Rancher (CNCF landscape) that utilizes the local filesystem of each worker node (`/opt/local-path-provisioner/`).
- **Why it is the ideal choice for your cluster:**

- **Zero Extra Footprint:** Consumes less than 30 MB of RAM across the entire cluster.
- **ARM64 Native:** Fully compatible with Apple Silicon architectures.
- **Dynamic Lifecycle Management:** When a pod requesting storage is scheduled to `k8s-worker-1`, the controller dynamically creates a isolated subfolder on `k8s-worker-1`, binds it to a dynamic `PersistentVolume`, mounts it into the container, and automatically cleans up the directory when the PVC is deleted.
- **Volume Binding Mode (WaitForFirstConsumer):** It intelligently waits until the Kubernetes scheduler decides which worker node will host the pod before creating the storage directory on that specific node.

How to Implement Phase 7.1

Phase 7.1 involves inspecting the Rancher Local Path Provisioner architecture and verifying that your worker nodes have the necessary directories and permissions ready.

Step 1: Verify Host Filesystem Permissions

The provisioner uses `/opt/local-path-provisioner` on the worker nodes by default to store dynamic volumes. Verify that the root filesystem on both `k8s-worker-1` and `k8s-worker-2` has writable `/opt` partitions.

Run from your **Mac terminal**:

```
orb -m k8s-worker-1 df -h /opt
orb -m k8s-worker-2 df -h /opt
```

Verification check: Both nodes will show available disk space shared from your Mac's internal virtual disk.

Step 2: Review the Provisioner Manifest Components

Before applying in 7.2, examine what the provisioner manifest creates:

```
curl -sSL https://raw.githubusercontent.com/rancher/local-path-provisioner/v0.0.30/deploy/local-path-storage
```

Verification check: The output lists the core resources:

- **Namespace:** `local-path-storage`
- **ServiceAccount:** Controller identity
- **ClusterRole & ClusterRoleBinding:** Permissions to watch PVCs and create/delete PVs
- **ConfigMap:** Defines the default local path (`/opt/local-path-provisioner`) and helper pod scripts
- **Deployment:** The controller managing volume lifecycles
- **StorageClass:** Named `local-path`

User prompt: explain 7.2, why it is used and how to do that?

Response: **Phase 7.2 is deploying the Rancher Local Path Provisioner manifests and setting its StorageClass as the cluster-wide default.**

What Is It and Why Is It Used?

- **Deploying the Provisioner:** Applying the manifest installs a controller deployment (`local-path-provisioner`) into a dedicated namespace (`local-path-storage`). This controller continuously watches the Kubernetes API server for `PersistentVolumeClaim` (PVC) objects that request the `local-path` storage class.
- **Why Mark It as Default (`storageclass.kubernetes.io/is-default-class: "true"`):** By default, Kubernetes does not have a default storage class configured. If an application manifest, Helm chart, or stateful deployment (like Redis, Postgres, or Prometheus) defines a `PersistentVolumeClaim` without explicitly specifying the `storageClassName` attribute, Kubernetes rejects the claim or leaves it in an unbound, pending state forever. Setting `local-path` as the default ensures that any generic stateful workload immediately gets dynamic persistent storage out of the box.

How the Local Path Provisioner Operates

1. A user or workload applies a PVC requesting a volume (e.g., 1Gi).
2. Because the provisioner uses `volumeBindingMode: WaitForFirstConsumer`, the storage is **not** immediately provisioned upon PVC creation.
3. Instead, the storage driver waits until a Pod referencing that PVC is scheduled onto a specific node (e.g., `k8s-worker-1`).
4. Once scheduled, the provisioner spins up a temporary helper pod on `k8s-worker-1` to create a dedicated directory: `/opt/local-path-provisioner/<pvc-name>_<namespace>_<pv-name>/`
5. The provisioner creates a corresponding `PersistentVolume` (PV) object and binds it to the PVC.
6. The pod starts, with the directory mounted cleanly inside its container.

How to Implement It

Run these commands directly from your **Mac terminal**.

Step 1: Deploy the Provisioner Manifests

Apply the official deployment manifest directly from the Rancher repository:

```
kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-provisioner/v0.0.30/deploy/local-pat
```

Step 2: Verify the Controller Deployment

Check that the provisioner pod transitions to Running:

```
kubectl get pods -n local-path-storage -o wide
```

Verification check: The pod `local-path-provisioner-...` must display **STATUS: Running** and 1/1 containers ready.

Step 3: Inspect the StorageClass

Verify that the `local-path` storage class has been created:

```
kubectl get storageclass
```

Verification check: You will see `local-path` listed with provisioner `rancher.io/local-path` and volume binding mode `WaitForFirstConsumer`.

Step 4: Set Local-path as the Default StorageClass

Annotate the `StorageClass` to mark it as the default provisioner for the entire cluster:

```
kubectl patch storageclass local-path -p '{"metadata": {"annotations":{"storageclass.kubernetes.io/is-defa
```

Step 5: Verification

Run `kubectl get storageclass` again to confirm the default designation:

```
kubectl get storageclass
```

Verification check: The output must display `(default)` next to `local-path`:

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION
local-path (default)	rancher.io/local-path	Delete	WaitForFirstConsumer	false

User prompt: explain 7.3, why it is used and how to do that?

Response: **Phase 7.3 is deploying a stateful test workload to validate dynamic volume provisioning, host directory creation, and data persistence across pod restarts.**

What Is It and Why Is It Used?

Configuring the Local Path Provisioner and setting it as the default `StorageClass` in Phase 7.2 only sets up the control plane logic. Until you exercise the storage driver with an actual workload, you cannot be certain that:

- Dynamic Provisioning Functions:** The provisioner can successfully capture a `PersistentVolumeClaim` (PVC) and dynamically create a matching `PersistentVolume` (PV) without manual admin intervention.
- WaitForFirstConsumer Works:** Storage allocation is correctly deferred until the Pod is placed on a specific worker node (`k8s-worker-1` or `k8s-worker-2`).
- Helper Pod Execution:** The provisioner can successfully spin up its temporary creation/cleanup pods to create directory structures under `/opt/local-path-provisioner/` on the worker node.
- Data Persists Across Restarts:** Data written by a container survives container termination and pod recreation on that same node.

How to Do It

Run these commands from your **Mac terminal**.

Step 1: Create a Test PVC and Pod Manifest

Create a test file named `storage-test.yaml`:

```
cat <<EOF > storage-test.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: local-path-pvc
spec:
```

```

    accessModes:
      - ReadWriteOnce
    resources:
      requests:
        storage: 128Mi
---
apiVersion: v1
kind: Pod
metadata:
  name: volume-test-pod
spec:
  containers:
    - name: test-container
      image: busybox:1.36
      command: ["sh", "-c", "echo 'Hello from persistent storage' > /data/test.txt && sleep 3600"]
      volumeMounts:
        - name: persistent-storage
          mountPath: /data
  volumes:
    - name: persistent-storage
      persistentVolumeClaim:
        claimName: local-path-pvc
EOF

```

Note: The PVC does not specify storageClassName. Because you configured local-path as the default class in Phase 7.2, Kubernetes automatically assigns it.

Step 2: Apply and Observe Delayed Binding

Apply the manifest:

```
kubectl apply -f storage-test.yaml
```

Check the PVC and Pod status:

```
kubectl get pvc local-path-pvc
kubectl get pods volume-test-pod -o wide
```

Verification check:

- Initially, you may see the PVC in Pending until the scheduler assigns volume-test-pod to a worker node.
- Once scheduled to a worker (e.g., k8s-worker-1), the PVC transitions to **Bound**.
- The Pod volume-test-pod transitions to **Running**.

Step 3: Verify Dynamic PV Creation and Node Binding

Inspect the dynamically generated PersistentVolume:

```
kubectl get pv
```

Verification check:

- A PV named pvc-<uuid> is automatically listed with STATUS: Bound and CLAIM: default/local-path-pvc.
- Check the volume's node affinity:

```
kubectl get pv $(kubectl get pvc local-path-pvc -o jsonpath='{.spec.volumeName}') -o jsonpath='{.spec.
```

It will show a required affinity matching the specific worker node where the pod was placed.

Step 4: Verify Data on the Worker Host Filesystem

Identify which worker node is running the pod:

```
NODE=$(kubectl get pod volume-test-pod -o jsonpath='{.spec.nodeName}')
echo "Pod is on $NODE"
```

Inspect the host directory inside that worker VM:

```
orb -m $NODE sudo ls -la /opt/local-path-provisioner/
orb -m $NODE sudo cat /opt/local-path-provisioner/*/test.txt
```

Verification check: The command will output:

```
Hello from persistent storage
```

This confirms that the host volume was dynamically created and populated.

Step 5: Test Persistence Across Pod Deletion

Delete the Pod (leaving the PVC intact):

```
kubectl delete pod volume-test-pod
```

Recreate only the Pod pointing to the same PVC:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: volume-test-pod-2
spec:
  containers:
  - name: test-container
    image: busybox:1.36
    command: ["sh", "-c", "cat /data/test.txt && sleep 3600"]
    volumeMounts:
    - name: persistent-storage
      mountPath: /data
  volumes:
  - name: persistent-storage
    persistentVolumeClaim:
      claimName: local-path-pvc
EOF
```

Check the logs of the new pod:

```
kubectl wait --for=condition=Ready pod/volume-test-pod-2 --timeout=60s
kubectl logs volume-test-pod-2
```

Verification check: The output must read Hello from persistent storage.

Step 6: Cleanup Test Resources

Delete the test resources:

```
kubectl delete pod volume-test-pod-2
kubectl delete pvc local-path-pvc
rm -f storage-test.yaml
```

Verify that the PV was cleaned up:

```
kubectl get pv
```

Verification check: Because the reclaim policy is Delete, the PV disappears and the Local Path Provisioner automatically cleans up the directory on the worker VM.

```
User prompt: macbeee@macbees-mac In-House_k8s_setup % kubectl apply -f storage-test.yaml persistentvolumeclaim/local-path-pvc created
pod/volume-test-pod created macbeee@macbees-mac In-House_k8s_setup % kubectl get pvc local-path-pvc NAME STATUS
VOLUME CAPACITY ACCESS MODES STORAGECLASS VOLUMEATTRIBUTESCLASS AGE local-path-pvc Pending
local-path <unset> 7s macbeee@macbees-mac In-House_k8s_setup % kubectl get pvc local-path-pvc NAME
STATUS VOLUME CAPACITY ACCESS MODES STORAGECLASS VOLUMEATTRIBUTESCLASS AGE local-path-pvc Pending
local-path <unset> 13s macbeee@macbees-mac In-House_k8s_setup % kubectl get pods volume-test-pod -o
wide NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES volume-test-pod 0/1
Pending 0 19s <none> <none> <none> <none> macbeee@macbees-mac In-House_k8s_setup % kubectl get pv NAME
CAPACITY ACCESS MODES RECLAIM POLICY STATUS CLAIM STORAGECLASS
VOLUMEATTRIBUTESCLASS REASON AGE pvc-cc1c572f-36a0-4b0f-960e-e71963e80289 128Mi RWO Delete Bound
default/local-path-pvc local-path <unset> 20s macbeee@macbees-mac In-House_k8s_setup % kubectl get pv $(kubectl get
pvc local-path-pvc -o jsonpath='{.spec.volumeName}') -o jsonpath='{.spec.nodeAffinity}' {"required":{"nodeSelectorTerms":[{"matchExpressions":
[{"key":"kubernetes.io/hostname","operator":"In","values":["k8s-worker-1"]}]}]}%
macbeee@macbees-mac In-House_k8s_setup % NODE=$(kubectl get pod volume-test-pod -o jsonpath='{.spec.nodeName}') echo "Pod is on
$NODE" Pod is on k8s-worker-1 macbeee@macbees-mac In-House_k8s_setup % orb -m $NODE sudo ls -la /opt/local-path-provisioner/ total 0
drwxr-xr-x 1 root root 126 Sep 13 02:41 . drwxr-xr-x 1 root root 98 Sep 13 02:41 .. drwxrwxrwx 1 root root 16 Sep 13 02:41 pvc-cc1c572f-36a0-
4b0f-960e-e71963e80289_default_local-path-pvc macbeee@macbees-mac In-House_k8s_setup % orb -m $NODE pwd
/Users/macbeee/again/Projects/In-House_k8s_setup macbeee@macbees-mac In-House_k8s_setup % orb -m $NODE sudo cat /opt/local-path-
provisioner/*/test.txt zsh: no matches found: /opt/local-path-provisioner/*/test.txt macbeee@macbees-mac In-House_k8s_setup %
worker-1 sudo cat /opt/local-path-provisioner/*/test.txt zsh: no matches found: /opt/local-path-provisioner/*/test.txt macbeee@macbees-mac In-
House_k8s_setup %
```

Response: The error zsh: no matches found: /opt/local-path-provisioner/*/test.txt happened because of two things:

- Zsh wildcard expansion:** Your local Mac shell (zsh) tried to expand * locally on your Mac before passing it to orb. Quoting the remote command prevents macOS zsh from attempting local path expansion.
- Pod Container Status:** The pod was still pulling busybox:1.36 or starting up when you checked, meaning the echo ... > /data/test.txt command hadn't executed yet.

Step 1: Check Pod Status

Run from your Mac:

```
kubectl get pod volume-test-pod
```

Verification check: The status should show **Running**. If it still shows **ContainerCreating**, wait a few seconds for the image pull to finish.

Step 2: Read the File with Escaped/Quoted Wildcards

Wrap the remote command in quotes so orb passes the wildcard directly to the VM's bash shell:

```
orb -m k8s-worker-1 'sudo cat /opt/local-path-provisioner/*/test.txt'
```

Alternatively, specify the exact folder path that was created:

```
orb -m k8s-worker-1 sudo cat /opt/local-path-provisioner/pvc-cc1c572f-36a0-4b0f-960e-e71963e80289_default_
```

Verification check: The output will display:

```
Hello from persistent storage
```

Step 3: Verify Data from Inside the Pod

You can also verify directly through Kubernetes without touching the VM host:

```
kubectl exec volume-test-pod -- cat /data/test.txt
```

Verification check: It outputs **Hello from persistent storage**.

```
User prompt: macbeee@macbees-mac In-House_k8s_setup % kubectl get pv NAME CAPACITY ACCESS MODES
RECLAIM POLICY STATUS CLAIM STORAGECLASS VOLUMEATTRIBUTESCLASS REASON AGE pvc-cc1c572f-36a0-
4b0f-960e-e71963e80289 128Mi RWO Delete Bound default/local-path-pvc local-path <unset> 12m
macbeee@macbees-mac In-House_k8s_setup %
```

Response: Your dynamic volume is successfully provisioned and bound. Now you can verify data persistence across pod deletion (Steps 4–6 of Phase 7.3).

Step 1: Verify the File via `kubectl exec`

Check that the container wrote the test file directly:

```
kubectl exec volume-test-pod -- cat /data/test.txt
```

Verification check: The terminal should output:

```
Hello from persistent storage
```

Step 2: Delete the Pod (Keep the PVC)

Delete the pod to test whether the volume survives container destruction:

```
kubectl delete pod volume-test-pod
```

Verify that the PVC and PV remain **Bound**:

```
kubectl get pvc local-path-pvc
kubectl get pv
```

Verification check: Both will still show **STATUS: Bound**. The underlying directory on `k8s-worker-1` is preserved.

Step 3: Recreate a Second Pod to Read the Persistent Data

Launch a new pod mounting the exact same PVC:

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: volume-test-pod-2
spec:
  containers:
  - name: test-container
    image: busybox:1.36
    command: ["sh", "-c", "cat /data/test.txt && sleep 3600"]
    volumeMounts:
    - name: persistent-storage
      mountPath: /data
  volumes:
  - name: persistent-storage
    persistentVolumeClaim:
      claimName: local-path-pvc
EOF
```

Wait until it starts, then inspect its logs:

```
kubectl wait --for=condition=Ready pod/volume-test-pod-2 --timeout=60s
kubectl logs volume-test-pod-2
```

Verification check: The output must print:

Hello from persistent storage

Step 4: Cleanup Test Resources

Once persistence is confirmed, clean up the test resources:

```
kubectl delete pod volume-test-pod-2
kubectl delete pvc local-path-pvc
rm -f storage-test.yaml
```

Check the PersistentVolume list:

```
kubectl get pv
```

Verification check: The PV will be automatically removed (No resources found) because the reclaim policy is Delete, and the Local Path Provisioner will clean up the directory on k8s-worker-1.

User prompt: explain phase 8, why it is used and how to do that?

Response: **Phase 8 is end-to-end cluster validation.** It verifies that your networking data plane, cross-node overlay encapsulation, Service routing engine, and cluster DNS work cohesively before deploying real applications or running zero-downtime upgrades.

Why Phase 8 Is Necessary

A cluster can show all nodes in Ready status and still have critical underlying networking flaws:

1. **Cross-Node Overlay Routing (VXLAN):** Flannel creates tunnel interfaces (flannel.1) to encapsulate pod traffic across nodes. If firewall rules block UDP port 8472 or if MTU negotiation fails, pods can ping local pods on the same node but fail completely when talking to pods on another node.
2. **ClusterIP Service Translation (kube-proxy):** Validates that iptables rules programmed by kube-proxy properly perform DNAT on virtual ClusterIP addresses and distribute connections to healthy backend Pod endpoints.
3. **Internal Service Discovery (CoreDNS):** Validates that pods receive /etc/resolv.conf configurations pointing to the CoreDNS service IP (10.96.0.10) and can resolve both internal Kubernetes domain names (<service>.<namespace>.svc.cluster.local) and external internet domains.

Step-by-Step Implementation

Run these validation steps directly from your **Mac terminal**.

Step 1: Deploy Workloads Across Separate Nodes

Deploy a two-replica Nginx deployment with a pod anti-affinity rule to guarantee that the pods land on **different** worker nodes (k8s-worker-1 and k8s-worker-2), fronted by a ClusterIP service:

```
cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: net-check-nginx
  labels:
    app: net-check
spec:
  replicas: 2
  selector:
    matchLabels:
      app: net-check
  template:
    metadata:
      labels:
        app: net-check
    spec:
      affinity:
        podAntiAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            - labelSelector:
                matchExpressions:
                  - key: app
                    operator: In
                    values: ["net-check"]
              topologyKey: "kubernetes.io/hostname"
```

```

    containers:
    - name: nginx
      image: nginx:1.27-alpine
      ports:
      - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: net-check-svc
spec:
  selector:
    app: net-check
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
EOF

```

Verify pod distribution across the nodes:

```
kubectl get pods -l app=net-check -o wide
```

Verification check: One pod must run on k8s-worker-1 and the other on k8s-worker-2, both in Running status with distinct IP addresses in the 10.244.x.x range.

Step 2: Spin Up an Interactive Diagnostic Client

Deploy an ephemeral client pod loaded with network utilities (curl, drill, iproute2):

```

cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: net-client
spec:
  containers:
  - name: client
    image: nicolaka/netshoot:latest
    command: ["sleep", "3600"]
EOF

```

Wait until it enters Running state:

```
kubectl wait --for=condition=Ready pod/net-client --timeout=60s
```

Step 3: Test Cross-Node Direct Pod-to-Pod Traffic

Obtain the pod IPs and the node running net-client:

```
kubectl get pod net-client -o wide
kubectl get pods -l app=net-check -o wide
```

Pick the Nginx pod that is **on a different node** than net-client. Ping and curl its direct Pod IP from inside net-client:

```

TARGET_POD_IP=$(kubectl get pods -l app=net-check -o jsonpath='{.items[0].status.podIP}')
kubectl exec net-client -- ping -c 3 $TARGET_POD_IP
kubectl exec net-client -- curl -s -I http://$TARGET_POD_IP

```

Verification check: The ping returns 0% packet loss and curl returns HTTP/1.1 200 OK. This proves Flannel's VXLAN packet encapsulation is functioning across VM boundaries.

Step 4: Test Service Routing and Load Balancing (kube-proxy)

Query the Service name directly. kube-proxy should intercept the request and route to the backend pods:

```
kubectl exec net-client -- curl -s -I http://net-check-svc
```

Verification check: Returns HTTP/1.1 200 OK.

Step 5: Test DNS Resolution (CoreDNS)

Test internal name resolution, fully qualified domain name (FQDN) lookup, and external public DNS resolution from net-client:

1. Internal Service short-name resolution:

```
kubectl exec net-client -- drill net-check-svc
```

Verification check: The ANSWER SECTION lists the ClusterIP of the net-check-svc.

2. Full cluster FQDN resolution:

```
kubectl exec net-client -- drill net-check-svc.default.svc.cluster.local
```

Verification check: Resolves accurately to the same ClusterIP.

3. External upstream DNS forwarding:

```
kubectl exec net-client -- drill google.com
```

Verification check: Resolves public IP addresses, confirming CoreDNS forwarders match `/etc/resolv.conf` upstream rules.

Step 6: Cleanup Validation Resources

Once all tests pass, delete the verification fixtures:

```
kubectl delete deployment net-check-nginx
kubectl delete service net-check-svc
kubectl delete pod net-client
```
